

Java Exception Handling

1. What is an Exception?

An **exception** is an abnormal condition that occurs during program execution and disrupts the normal flow of the program.

Example:

```
int a = 10;  
int b = 0;  
  
int result = a / b;
```

This causes:

java.lang.ArithmeticException: / by zero

Instead of continuing normally, Java creates an exception object and looks for a mechanism to handle it.

Interview answer

An exception is an abnormal event that occurs during program execution and interrupts the normal flow of the application.

2. What is Exception Handling?

Exception handling is the mechanism used to **detect, handle, and recover from exceptional situations** so that the application can continue gracefully.

Example:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

Output:

Cannot divide by zero

Without handling:

Exception in thread "main"
java.lang.ArithmeticException: / by zero

3. Why do we need Exception Handling?

Main reasons:

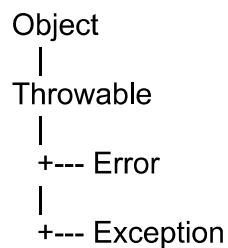
1. Prevent application crash.
2. Maintain normal program flow.
3. Provide meaningful error messages.
4. Separate business logic from error-handling logic.
5. Log errors for debugging.
6. Properly clean up resources.
7. In Spring Boot, return standardized HTTP error responses.

Interview answer

Exception handling allows us to handle unexpected situations gracefully instead of allowing the application to terminate abruptly.

4. What is the difference between Error and Exception?

Both are subclasses of **Throwable**.



Error

Generally represents serious problems that applications normally **should not try to recover from**.

Examples:

OutOfMemoryError
StackOverflowError
VirtualMachineError

Exception

Represents conditions that an application can often handle.

Examples:

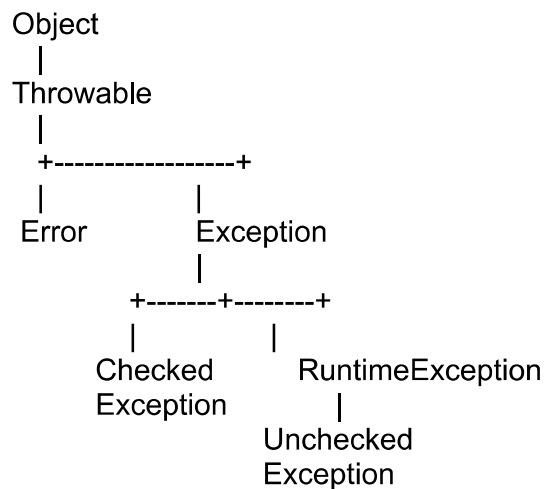
IOException
SQLException
NullPointerException
ArithmeticException

Interview answer

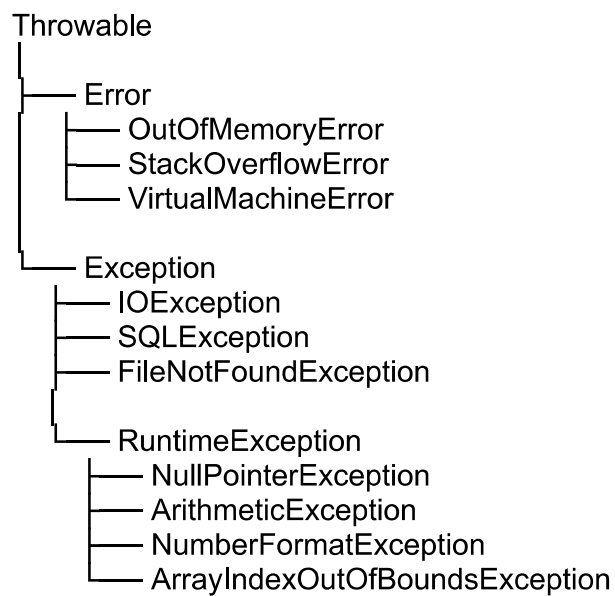
Error generally represents serious JVM/system-level problems, while **Exception** represents conditions that application code can usually handle.

5. What is the Exception Hierarchy?

The important hierarchy is:



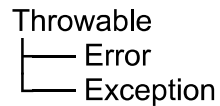
Examples:



6. What is **Throwable**?

Throwable is the **root class** for objects that can be thrown by Java's exception mechanism.

It has two major branches:



Important methods include:

```
getMessage()
printStackTrace()
toString()
```

7. What is an **Error**?

Error indicates serious problems generally related to the JVM or runtime environment.

Examples:

OutOfMemoryError

Occurs when JVM cannot allocate required memory.

StackOverflowError

Usually occurs because of excessive recursion.

```
public void test() {
    test();
}
```

VirtualMachineError

Represents serious JVM problems.

Should we catch **Error?**

Normally:

```
catch (Error e)
```

is **not recommended** for normal application recovery.

8. What are the different types of errors?

Your diagram shows:

1. Syntax/Compile Error

Example:

```
int x = ;
```

The Java compiler detects it.

2. Logical Error

Program runs but produces incorrect output.

```
int salary = 10000;  
int bonus = 5000;
```

```
int total = salary - bonus; // logically wrong
```

No exception necessarily occurs.

3. Runtime Error

Problems occurring while the application is running.

Example:

```
int x = 10 / 0;
```

This produces:

ArithmeticException

Important correction

Logical errors and compile-time errors are not types of Exception.

This distinction is important in interviews.

9. What is a Checked Exception?

A checked exception is an exception that the compiler requires you to **handle or declare**.

Examples:

IOException
SQLException
FileNotFoundException

Example:

```
public void readFile() throws IOException {  
    FileReader reader = new FileReader("data.txt");  
}
```

You must either:

Handle it

```
try {  
    FileReader reader = new FileReader("data.txt");  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

or:

Declare it

```
public void readFile() throws IOException {  
}
```

10. What is an Unchecked Exception?

Unchecked exceptions are generally subclasses of:

RuntimeException

The compiler doesn't force you to catch or declare them.

Examples:

NullPointerException
ArithmeticException
NumberFormatException
ArrayIndexOutOfBoundsException
IllegalArgumentException

Example:

```
int result = 10 / 0;
```

No compiler error occurs, but at runtime:

ArithmeticException

11. Checked vs Unchecked Exception

Feature	Checked	Unchecked
Parent	Exception excluding RuntimeException	RuntimeException
Compiler enforcement	Yes	No
Handling required	Catch or declare	Not mandatory
Usually represents	Recoverable/external conditions	Programming/state errors
Example	IOException	NullPointerException

Interview answer

Checked exceptions are verified by the compiler and must be caught or declared, whereas unchecked exceptions are subclasses of RuntimeException and do not have that compiler requirement.

12. Is Checked Exception really a "Compile-Time Exception"?

Be careful with your diagram.

It says:

CHECKED
(COMPILED TIME)

Better terminology:

Checked exceptions are compile-time checked by the compiler.

The exception itself generally occurs at **runtime**.

For example:

```
FileReader reader = new FileReader("abc.txt");
```

The compiler says:

Unhandled exception: FileNotFoundException

But the actual file operation occurs at runtime.

13. What are the keywords used for Exception Handling?

Java provides:

```
try
catch
finally
throw
throws
```

14. What is **try**?

try contains code that may throw an exception.

```
try {
    int result = 10 / 0;
}
```

15. What is **catch**?

catch handles an exception thrown from the associated **try**.

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Division by zero");
}
```

16. What is **finally**?

finally contains code that generally executes whether an exception occurs or not.

```
try {
    System.out.println("Try");
} catch (Exception e) {
    System.out.println("Catch");
}
```



```
} finally {  
    System.out.println("Finally");  
}
```

Output:

Try
Finally

Common use:

Resource cleanup
Connection cleanup
File cleanup

Although today, **try-with-resources** is preferred for **AutoCloseable** resources.

17. Can we have **try** without **catch**?

Yes.

```
try {  
    System.out.println("Hello");  
} finally {  
    System.out.println("Cleanup");  
}
```

try must have either:

catch

or

finally

18. Can we have **catch** without **try**?

No.

Invalid:

```
catch (Exception e) {  
}
```

A **catch** must be associated with a **try**.

19. Can we have multiple catch blocks?

Yes.

```
try {  
    int result = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Arithmetic error");  
}  
catch (Exception e) {  
    System.out.println("Generic error");  
}
```

20. What is the order of multiple catch blocks?

Specific exception first, generic exception later.

Correct:

```
catch (ArithmeticException e) {  
}  
catch (RuntimeException e) {  
}  
catch (Exception e) {  
}
```

Incorrect:

```
catch (Exception e) {  
}  
catch (ArithmeticException e) {  
}
```

Because `Exception` already catches `ArithmeticException`.

The compiler will report unreachable code.

21. What is Multi-Catch?

Java allows multiple exception types in one catch.

```
try {  
    // code  
}
```

```
catch (IOException | SQLException e) {  
    e.printStackTrace();  
}
```

Useful when handling different exceptions in the same way.

22. What is **throw**?

throw is used to **explicitly throw an exception object**.

```
throw new RuntimeException("Something went wrong");
```

Example:

```
public void withdraw(double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("Invalid amount");  
    }  
}
```

Key point

throw is used to **actually throw** an exception.

23. What is **throws**?

throws is used in a method signature to **declare that a method may throw exceptions**.

```
public void readFile() throws IOException {  
}
```

It doesn't itself throw the exception.

24. **throw** vs **throws**

throw

Actually throws exception

Used inside method

throws

Declares possible exception

Used in method signature

Followed by exception object

Followed by exception class

One exception object at a time

Can declare multiple exceptions

Example:

```
throw new IOException();
```

vs.

```
public void test() throws IOException {  
}
```

25. What is Stack Trace?

A stack trace tells us **where an exception occurred and how the program reached that point.**

Example:

```
java.lang.ArithmeticException: / by zero  
    at com.example.Calculator.divide(Calculator.java:15)  
    at com.example.Service.process(Service.java:42)  
    at com.example.Controller.calculate(Controller.java:20)
```

It gives information such as:

- Exception type
 - Message
 - Class
 - Method
 - File
 - Line number
 - Call sequence
-

26. How can we print exception information?

Your diagram shows three approaches.

1. **println**

```
System.out.println(e);
```

Usually gives:

java.lang.ArithmeticException: / by zero

2. `getMessage()`

```
System.out.println(e.getMessage());
```

Gives only the message:

/ by zero

3. `printStackTrace()`

```
e.printStackTrace();
```

Provides detailed debugging information:

Exception type

Message

Class

Method

File

Line number

Call stack

Interview answer

`getMessage()` gives the exception message, `toString()` gives the exception type and message, and `printStackTrace()` gives the exception details along with the call stack.

27. What is the difference between `getMessage()` and `printStackTrace()`?

```
catch (Exception e) {
```

```
    System.out.println(e.getMessage());
```

```
    e.printStackTrace();
}
```

`getMessage()`:

/ by zero

`printStackTrace()`:

```
java.lang.ArithmeticException: / by zero
    at Calculator.divide(Calculator.java:10)
    ...
```

For production systems, use proper logging rather than directly using `printStackTrace()`.

28. What is Exception Propagation?

If an exception isn't handled in the current method, it propagates up the call stack.

Example:

```
main()
  ↓
service()
  ↓
repository()
  ↓
exception
```

If `repository()` doesn't handle it:

```
repository()
  ↓
service()
  ↓
main()
```

The JVM searches for a matching handler.

29. What happens if no one handles the exception?

The exception reaches the JVM's default exception handler.

The application/thread may terminate and the JVM prints a stack trace.

Example:

```
public static void main(String[] args) {
    int x = 10 / 0;
}
```

Output:

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
```

30. Can we create our own exception?

Yes.

Checked custom exception

```
public class UserNotFoundException extends Exception {  
  
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

Unchecked custom exception

```
public class UserNotFoundException extends RuntimeException {  
  
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

For most Spring Boot business/application exceptions, custom exceptions commonly extend `RuntimeException`.

31. Why do we create Custom Exceptions?

Instead of:

```
throw new RuntimeException("User not found");
```

we can use:

```
throw new UserNotFoundException("User not found");
```

This makes the code:

- More meaningful
 - Easier to maintain
 - Easier to handle globally
 - Easier to map to HTTP status codes
-

32. What is Exception Chaining?

One exception can have another exception as its cause.

```
try {
```

```
// database operation
}
catch (SQLException e) {
    throw new RuntimeException("Database operation failed", e);
}
```

Here:

```
RuntimeException
|
└─ caused by SQLException
```

This preserves the original cause.

33. What is `getCause()`?

Throwable cause = e.getCause();

It retrieves the underlying cause of an exception.

Example:

```
throw new RuntimeException("Service failed", sqlException);
```

Then:

```
e.getCause()
```

returns the original `SQLException`.

34. What is try-with-resources?

Introduced in Java 7.

It automatically closes resources implementing:

`AutoCloseable`

Example:

```
try (FileReader reader = new FileReader("data.txt")) {
    // read file
}
catch (IOException e) {
    e.printStackTrace();
}
```


The reader is automatically closed.

Before Java 7

```
FileReader reader = null;
```

```
try {  
    reader = new FileReader("data.txt");  
}  
finally {  
    if (reader != null) {  
        reader.close();  
    }  
}
```

Try-with-resources is cleaner and safer.

35. What happens if an exception occurs in **finally**?

This is dangerous.

```
try {  
    throw new RuntimeException("Try exception");  
}  
finally {  
    throw new RuntimeException("Finally exception");  
}
```

The exception from **finally** can replace/suppress the original exception in ordinary **try-finally** flow.

Therefore, avoid throwing exceptions unnecessarily from **finally**.

36. Does **finally** always execute?

The common interview answer is:

Normally, yes.

But there are exceptions.

For example:

```
System.exit(0);
```

can terminate the JVM before **finally** executes.

Other extreme JVM termination situations can also prevent it.

So don't say:

"finally ALWAYS executes."

Say:

"**finally** normally executes regardless of whether an exception occurs, except when execution of the JVM is terminated before it can run."

37. Can we override a method and change its exceptions?

Important interview question.

Suppose parent:

```
class Parent {  
    void test() throws IOException {  
    }  
}
```

Child:

```
class Child extends Parent {  
    @Override  
    void test() throws IOException {  
    }  
}
```

Allowed.

But:

```
@Override  
void test() throws Exception {  
}
```

Not allowed because **Exception** is broader than **IOException**.

A child method cannot throw a **broader checked exception** than the parent declaration.

It may throw:

- Same checked exception
- Subclass of checked exception

- No checked exception

Unchecked exceptions have different rules and aren't restricted this way.

38. Can `main()` throw an exception?

Yes.

```
public static void main(String[] args) throws Exception {  
}
```

This means `main()` doesn't handle it; it declares it.

39. Can a constructor throw an exception?

Yes.

```
public User() throws IOException {  
}
```

40. Can static methods throw exceptions?

Yes.

```
public static void load() throws IOException {  
}
```

41. Can we catch `Throwable`?

Technically yes:

```
catch (Throwable t) {  
}
```

But normally **don't do this**.

Why?

Because it catches both:

Exception
Error

including serious JVM problems.

Prefer:

```
catch (Exception e)
```

when you genuinely need a broad application-level catch.

42. Should we catch **Exception** everywhere?

No.

Bad:

```
try {  
    // everything  
}  
catch (Exception e) {  
    // ignore  
}
```

Problems:

- Hides bugs
- Makes debugging difficult
- Can swallow important failures
- May produce incorrect application behavior

Prefer catching the **most appropriate exception**.

43. What is Exception Swallowing?

This is when we catch an exception and do nothing meaningful with it.

Bad:

```
try {  
    process();  
}  
catch (Exception e) {  
}
```

The error disappears.

Better:

```
catch (Exception e) {
    log.error("Processing failed", e);
    throw e;
}
```

Or handle it appropriately.

44. Should we use exceptions for normal business logic?

Generally, no.

Bad:

```
try {
    findUser();
}
catch (UserNotFoundException e) {
    // expected normal flow
}
```

If something is genuinely expected and frequent, a normal return/result mechanism may be better.

Exceptions should represent exceptional situations.

45. What is Global Exception Handling in Spring Boot?

This is where your diagram becomes important.

Instead of doing:

```
@RestController
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {

        try {
            return service.getUser(id);
        }
        catch (Exception e) {
            // handle
        }
    }
}
```

in every controller, we create a centralized handler.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

}
```

46. What is **@ControllerAdvice**?

@ControllerAdvice provides **centralized cross-controller behavior**.

One of its most common uses is global exception handling.

Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(
        UserNotFoundException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

47. What is **@RestControllerAdvice**?

@RestControllerAdvice is effectively:

```
@ControllerAdvice
+
@ResponseBody
```

It is designed for REST APIs.

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

}
```

The returned object becomes the HTTP response body.

48. @ControllerAdvice vs @RestControllerAdvice

@ControllerAdvice	@RestControllerAdvice
Global controller advice	Global REST controller advice
Commonly used with MVC/views	Commonly used with REST APIs
May need @ResponseBody	Response body behavior included
Can handle exceptions	Can handle exceptions

For your Spring Boot REST API:

@RestControllerAdvice

is usually the better choice.

49. What is @ExceptionHandler?

@ExceptionHandler tells Spring:

"When this exception occurs, execute this method."

Example:

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<String> handleUserNotFound(
    UserNotFoundException ex) {

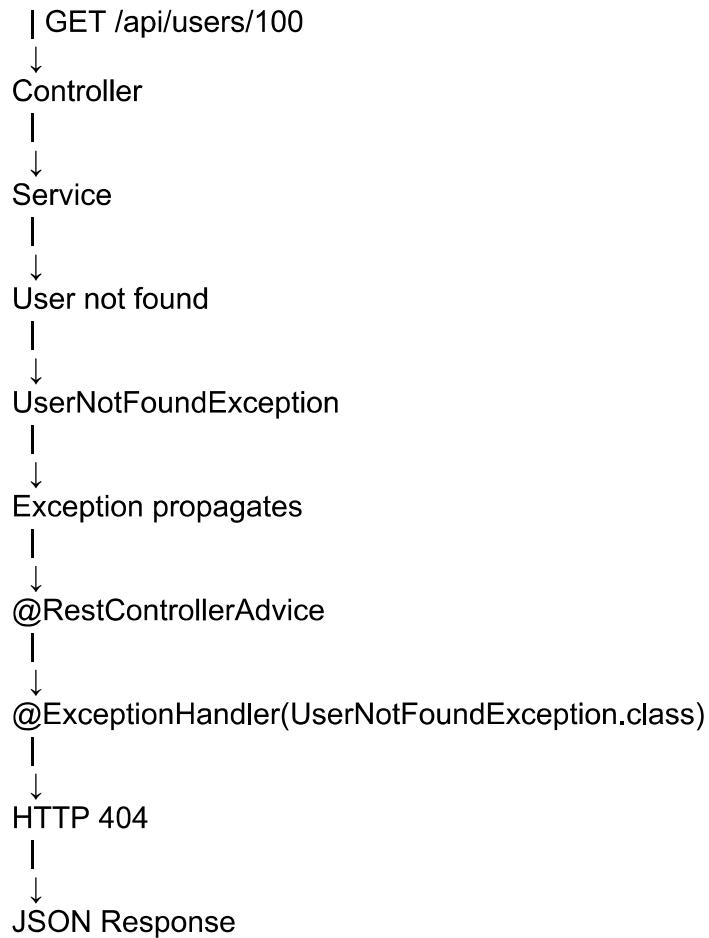
    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(ex.getMessage());
}
```

50. Explain the complete @ControllerAdvice flow

This is a **very important interview question**.

Suppose:

Client
|



Example response:

```
{
  "status": 404,
  "message": "User not found"
}
```

51. Complete Spring Boot Example

Custom Exception

```
public class UserNotFoundException extends RuntimeException {

    public UserNotFoundException(String message) {
        super(message);
    }
}
```

Service

```
@Service
```



```

public class UserService {

    public User getUser(Long id) {

        User user = repository.findById(id)
            .orElseThrow(() ->
                new UserNotFoundException(
                    "User not found with id: " + id
                )
            );

        return user;
    }
}

```

Controller

```

@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {

        return userService.getUser(id);
    }
}

```

Notice:

No try-catch.

Global Handler

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(
        UserNotFoundException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}

```

52. Can `@ExceptionHandler` handle multiple exceptions?

Yes.

```
@ExceptionHandler({
    UserNotFoundException.class,
    ProductNotFoundException.class
})
public ResponseEntity<String> handleNotFound(Exception ex) {

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(ex.getMessage());
}
```

Or:

```
@ExceptionHandler(UserNotFoundException.class)

for more precise handling.
```

53. Can we handle validation exceptions globally?

Yes.

For example:

```
@PostMapping
public User create(
    @Valid @RequestBody UserRequest request) {

    return service.create(request);
}
```

Validation failures can be handled globally using appropriate Spring exception handlers, such as:

MethodArgumentNotValidException

Example:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<?> handleValidation(
    MethodArgumentNotValidException ex) {

    Map<String, String> errors = new HashMap<>();

    ex.getBindingResult()
        .getFieldErrors()
        .forEach(error ->
            errors.put(
```

```

        error.getField(),
        error.getDefaultMessage()
    )
};

return ResponseEntity.badRequest().body(errors);
}

```

54. How do you handle 404, 400, 409, and 500 globally?

A professional REST API might have:

400 → Bad Request
 401 → Unauthorized
 403 → Forbidden
 404 → Resource Not Found
 409 → Conflict
 422 → Validation/Business issue depending on API design
 500 → Internal Server Error

Example:

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(
        UserNotFoundException ex) {

        ApiError error = new ApiError(
            404,
            ex.getMessage()
        );

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(error);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ApiError> handleBadRequest(
        IllegalArgumentException ex) {

        ApiError error = new ApiError(
            400,
            ex.getMessage()
        );

        return ResponseEntity

```

```
        .badRequest()
        .body(error);
    }
}
```

55. What is a Standard Error Response?

Instead of returning different formats:

"User not found"

and:

```
{
  "error": "Invalid user"
}
```

we can standardize:

```
{
  "timestamp": "2026-08-10T10:20:00",
  "status": 404,
  "error": "Not Found",
  "message": "User not found with id: 10",
  "path": "/api/users/10"
}
```

This is much better for frontend clients and debugging.

56. What is the difference between `@ExceptionHandler` inside Controller and `@ControllerAdvice`?

Local handling

```
@RestController
public class UserController {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<?> handle(...) {
    }
}
```

Only applies to that controller.

Global handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {
```

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<?> handle(...) {
}
}
```

Can apply across controllers.

Interview answer

A controller-level `@ExceptionHandler` handles exceptions locally for that controller, while `@ControllerAdvice` allows us to centralize exception handling across multiple controllers.

57. What happens if both Controller and ControllerAdvice have an ExceptionHandler?

Spring gives preference to a more local handler when applicable.

Conceptually:

Controller-specific handler
↓
Global ControllerAdvice
↓
More generic fallback

So controller-level handling can override broader global handling for that controller.

58. What if we have two `@ExceptionHandler`s that can handle the same exception?

Spring may face ambiguity if equally applicable handlers exist.

For example:

```
@ExceptionHandler(Exception.class)
```

and another equally applicable handler could cause ambiguity depending on placement/signatures.

The general principle is:

Define specific handlers for specific exceptions and keep a generic fallback handler last.

59. Should we use `printStackTrace()` in Spring Boot?

Generally **no**.

Instead use a logging framework:

```
private static final Logger log =  
    LoggerFactory.getLogger(GlobalExceptionHandler.class);
```

Then:

```
log.error("Unexpected error occurred", ex);
```

This preserves the stack trace in your logging system.

60. How do you handle unexpected exceptions?

Use a generic fallback:

```
@ExceptionHandler(Exception.class)  
public ResponseEntity<ApiError> handleException(Exception ex) {  
  
    log.error("Unexpected error", ex);  
  
    ApiError error = new ApiError(  
        500,  
        "Internal server error"  
    );  
  
    return ResponseEntity  
        .status(HttpStatus.INTERNAL_SERVER_ERROR)  
        .body(error);  
}
```

Important

Don't expose:

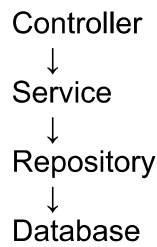
```
ex.getMessage()
```

blindly to clients for every unexpected exception.

It could reveal internal implementation details.

61. What is the best exception-handling architecture in a Spring Boot project?

For your backend projects, use:



and:



Project structure:

```
exception/  
├── GlobalExceptionHandler.java  
├── UserNotFoundException.java  
├── ProductNotFoundException.java  
├── DuplicateResourceException.java  
└── ApiError.java
```

62. What exceptions should Service Layer throw?

The service layer should throw **meaningful business/application exceptions**.

Example:

```
if (repository.existsByEmail(email)) {  
    throw new DuplicateResourceException(  
        "Email already exists"  
    );  
}
```

Don't unnecessarily convert every exception into:

RuntimeException

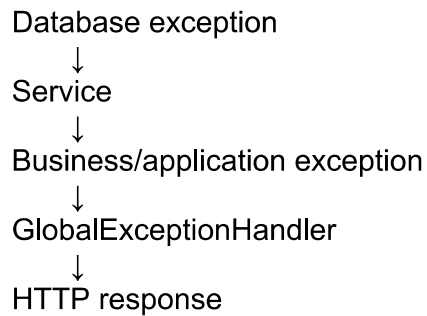
because that loses semantic information.

63. Should Repository handle exceptions?

Usually, the repository/data-access layer handles database interaction, while Spring Data translates many persistence exceptions into Spring's data-access exception hierarchy.

The service layer should decide what the business meaning is.

For example:



64. What is the difference between Exception and HTTP Error?

This is extremely important for backend interviews.

An exception is a **Java/application concept**:

UserNotFoundException

HTTP status is an **API/protocol concept**:

404 NOT FOUND

We map:

UserNotFoundException

↓
HTTP 404

using:

@ExceptionHandler

or other Spring mechanisms.

65. Can we use @ResponseStatus?

Yes.

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotFoundException
```



```

    extends RuntimeException {

    public UserNotFoundException(String message) {
        super(message);
    }
}

```

Then throwing:

```
throw new UserNotFoundException("User not found");
```

can result in HTTP 404.

However, centralized `@RestControllerAdvice` is often preferred when you need:

- Standard response structure
- Logging
- Different handling rules
- Consistent API errors
- More control

66. `@ResponseStatus` vs `@ExceptionHandler`

`@ResponseStatus`

Simple:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
```

`@ExceptionHandler`

More flexible:

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ApiError> handle(...) {
}

```

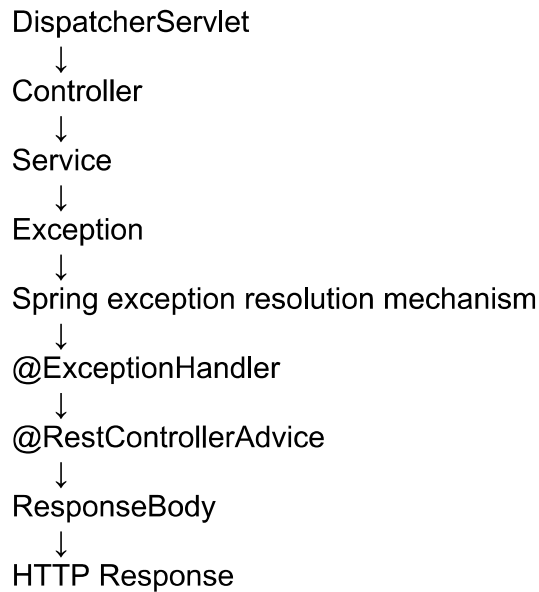
For production REST APIs, centralized exception handling usually gives more control.

67. Advanced: What happens when an exception occurs in Spring MVC?

Simplified flow:

HTTP Request

↓



DispatcherServlet coordinates request processing and Spring's exception resolution mechanisms determine how the exception should be handled.

68. Advanced: What is **ResponseEntityExceptionHandler**?

Spring provides:

ResponseEntityExceptionHandler

which can be extended to customize handling of many Spring MVC exceptions.

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler
    extends ResponseEntityExceptionHandler {

}
```

This is useful when you want centralized customization for framework-level exceptions.

69. Advanced: What is **ProblemDetail**?

Modern Spring applications can use the standardized HTTP API error representation based on:

ProblemDetail

Example:

```
@ExceptionHandler(UserNotFoundException.class)
public ProblemDetail handleUserNotFound(
    UserNotFoundException ex) {

    ProblemDetail problem =
        ProblemDetail.forStatus(
            HttpStatus.NOT_FOUND
        );

    problem.setTitle("User Not Found");
    problem.setDetail(ex.getMessage());

    return problem;
}
```

This is a more standardized approach to REST error responses.

70. What is the most important difference between **throw** and **throws**?

Remember this:

throw → actually throw
throws → declare

Example:

```
throw new UserNotFoundException("User not found");
```

versus:

```
public User findUser(Long id)
    throws UserNotFoundException {
}
```

71. What happens when you throw a RuntimeException?

Example:

```
public void test() {

    throw new RuntimeException("Failed");

    // code here will not execute
}
```

The current method stops executing and the exception propagates upward until a matching handler is found.

72. What happens after a catch block executes?

Once a matching **catch** handles the exception, normal execution continues **after the entire try-catch-finally construct**, unless the catch/finally itself throws or returns.

Example:

```
try {  
    int x = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Handled");  
}
```

```
System.out.println("Continue");
```

Output:

```
Handled  
Continue
```

73. What happens if the catch block throws another exception?

Example:

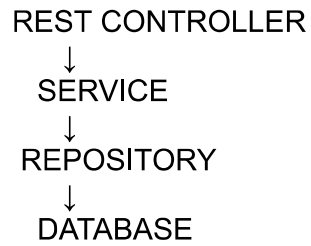
```
try {  
    int x = 10 / 0;  
}  
catch (ArithmeticException e) {  
    throw new RuntimeException("New exception");  
}
```

The new exception propagates from the catch block.

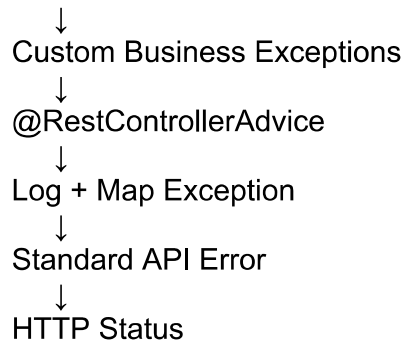
74. What is a good production exception-handling strategy?

For a Spring Boot backend:

```
CLIENT  
↓
```



Exceptions



Example:

```
{  
  "timestamp": "2026-08-10T10:30:00",  
  "status": 404,  
  "error": "NOT_FOUND",  
  "message": "Product not found with id: 101",  
  "path": "/api/products/101"  
}
```

🔥 Top 20 Interview Questions You MUST Know

If you're preparing for a **Java/Spring Boot backend interview**, prioritize these:

#	Question	Priority
1	What is an exception?	★★★★★
2	Why do we need exception handling?	★★★★★
3	Explain exception hierarchy	★★★★★
4	Error vs Exception	★★★★★
5	Checked vs unchecked exception	★★★★★
6	throw vs throws	★★★★★

7	<code>try-catch-finally</code>	★★★★★
8	Can try exist without catch?	★★★★
9	Can catch exist without try?	★★★★
1 0	Multiple catch blocks	★★★★
11	Exception propagation	★★★★★
1 2	Custom exception	★★★★★
1 3	Exception chaining	★★★★
1 4	<code>getMessage()</code> vs <code>printStackTrace()</code>	★★★★
1 5	try-with-resources	★★★★★
1 6	What is <code>@ControllerAdvice</code> ?	★★★★★
1 7	What is <code>@RestControllerAdvice</code> ?	★★★★★
1 8	What is <code>@ExceptionHandler</code> ?	★★★★★
1 9	Explain global exception handling flow	★★★★★
2 0	Design exception handling in Spring Boot	★★★★★

One-Minute Interview Revision

If the interviewer says:

"Explain Exception Handling in Java."

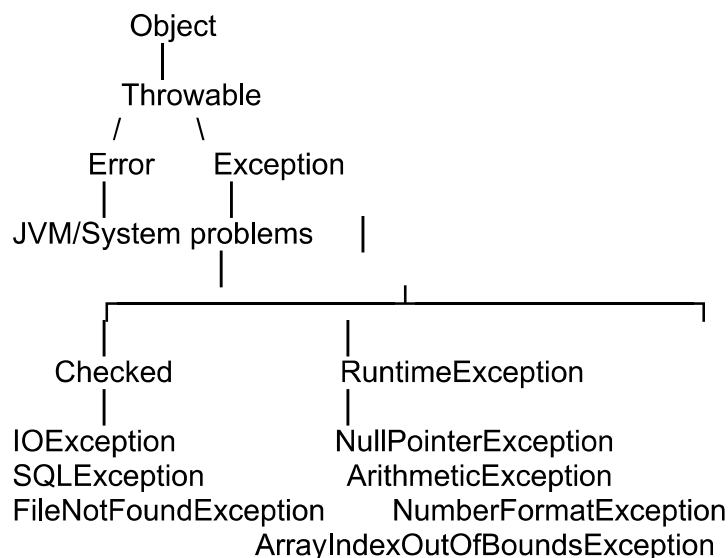
You can answer:

Exception handling in Java is a mechanism to handle abnormal conditions during program execution without abruptly terminating the application. The root class is `Throwable`, which has two major subclasses: `Error` and `Exception`. Errors generally represent

serious JVM-level problems, while exceptions represent application-level problems. Exceptions are mainly classified into checked and unchecked exceptions. Checked exceptions are checked by the compiler and must be caught or declared, while unchecked exceptions extend `RuntimeException` and don't have that requirement. Java provides `try`, `catch`, `finally`, `throw`, and `throws` for exception handling. We can also create custom exceptions and use exception chaining and try-with-resources. In Spring Boot REST applications, we commonly use `@RestControllerAdvice` with `@ExceptionHandler` to implement centralized global exception handling and convert Java exceptions into standardized HTTP responses.

The complete mental model

Memorize this diagram:

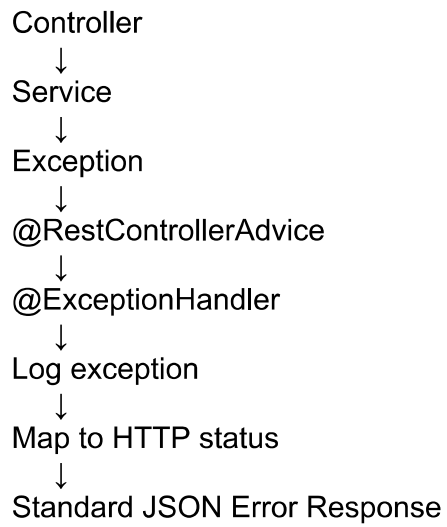


Java Handling

```
try
{
  exception → catch
  always → finally
}
```

`throw` → explicitly throw exception
`throws` → declare exception

Spring Boot



This is the **core exception-handling chain** you should be able to explain on a **whiteboard** in an interview.